Protocol-Governed Systems: Runtime Conceptual Model

Contact: mailto:bachipeachy@gmail.com

ORCID Profile: https://orcid.org/0009-0007-3810-6520

Preface

This paper is part of the PGS technical paper series. The paper *Protocol-Governed Systems: Conceptual Model* established the architectural foundations: constitutional governance, the four-layer stack, and the separation of governance from execution. The paper *Protocol-Governed Systems: Compiler Conceptual Model* described how the compiler converts protocol declarations into a governed execution boundary called the Protocol Snapshot. Together, those two papers establish that behavior is fully determined before execution begins. This paper focuses on the component that consumes that boundary: the PGS runtime.

The runtime occupies the narrowest part of the PGS architecture. The compiler determines what may exist. The runtime determines what happens when existence is realized. Understanding what the runtime does, what it deliberately does not do, and why those boundaries are load-bearing is essential to understanding how Protocol-Governed Systems achieve governed execution in practice.

Abstract

Protocol-Governed Systems (PGS) separate governance from execution. The compiler produces a Protocol Snapshot that encodes every admissible execution path as a compiled topology. The runtime consumes that snapshot and executes workflow instances. The runtime has no knowledge of protocol declarations, governance rules, or constitutional boundaries. It traverses only what the compiler constructed.

This paper defines the conceptual model of the PGS runtime. It explains how execution can exist independently of implementation, how a single compiled snapshot can execute identically across radically different hosting substrates, and why runtime simplicity is the strongest possible proof that governance is complete. The paper introduces the core runtime properties: implementation independence, substrate independence, hosting transparency, projection independence, security as a projection choice, runtime multiplicity, transport neutrality, structural parallelism, runtime stability, and trace portability. It also defines the runtime's constitutional boundary: what the runtime knows, what it is structurally forbidden from knowing, and why these boundaries are load-bearing.

1. Introduction

Every software system must eventually execute. Protocol-Governed Systems are no exception. After governance declarations are authored and the compiler has constructed an admissible execution surface, something must traverse that surface when a workflow invocation arrives. That something is the runtime.

Most execution engines embed knowledge. They contain routing logic, business rules, fallback paths, conditional behavior, and interpretation layers. The larger and more complex a system becomes, the more this embedded knowledge diverges from the declared intent. This divergence is one of the primary failure modes that PGS addresses.

The PGS runtime takes the opposite approach. It contains no domain knowledge. It interprets nothing. It infers nothing. It falls back to nothing. Every routing decision, every input binding, every capability invocation, every permissible outcome — all of it is declared in the compiled snapshot. The runtime's job is to read those declarations and execute them faithfully.

This creates a striking simplicity. The runtime does not know what a wallet is, what an actor is, what a blockchain is, or what AI governance means. It knows how to traverse a compiled topology, invoke declared capabilities, and produce structured execution evidence. It applies that same procedure to every workflow in every domain.

> **The compiler governs possibility. The runtime governs realization.**

This separation is not a design preference. It is a constitutional requirement. If the runtime contained domain logic, governance would be shared between the protocol and the implementation. Protocol sovereignty — the principle that protocol is the sole source of behavioral truth — would be violated.

But the separation has a deeper consequence that this paper explores. Because the runtime contains no domain logic, execution becomes portable. The same snapshot can be consumed by any runtime that implements the execution contract. This is the truly novel contribution: **execution can exist independently of implementation.**

2. Implementation Independence

The compiler paper established *governance portability*: protocol governance can be declared, compiled, and verified before any execution substrate exists.

This paper establishes the complementary property: **implementation independence** — protocol behavior can be executed without coupling to a specific implementation.

In traditional systems, behavior is embedded in implementation:

```
Application Code = Business Logic + Routing + Implementation + Execution Repr
esentation
```

To change the behavior, you change the code. To port to a new substrate, you rewrite or adapt the code. To audit what the system does, you must read the implementation. Behavior and implementation are one.

PGS separates them:

```
Protocol        = Behavior (implementation-independent)
Snapshot        = Compiled behavior projected into execution form
Runtime         = Executor of compiled behavior (domain-ignorant)
Implementation  = Replaceable substrate
```

This separation has a concrete consequence: **the implementation can be replaced entirely without changing the protocol behavior.** A new runtime can be written for a new substrate — embedded firmware, FPGA, browser, serverless — and it executes the same protocol with the same outcomes, the same routing, the same governance.

The inverse is equally important: **the protocol can evolve entirely without changing the runtime.** New domains, new workflows, new governance rules, new capability contracts — all compiled into a new snapshot, all executed by the same runtime.

Implementation independence is not a portability feature layered on top of the system. It is the structural consequence of putting behavior in the protocol and keeping the runtime ignorant of it.

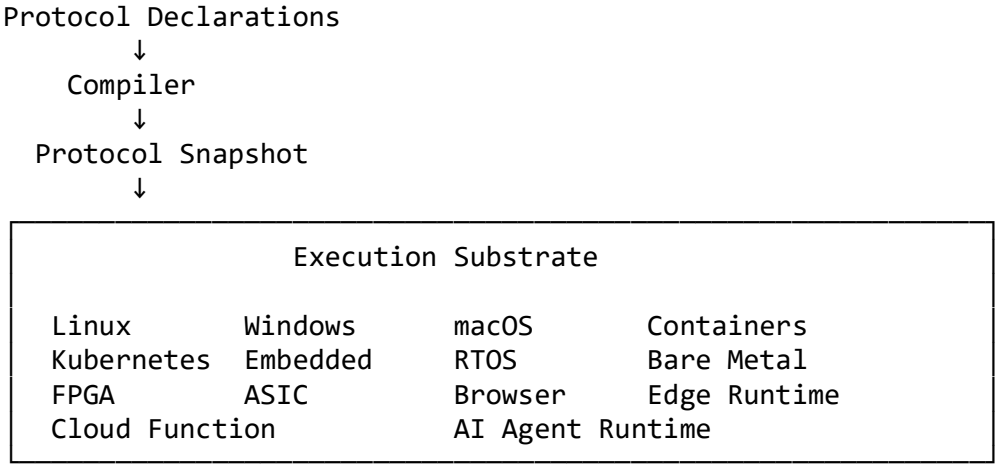
> **Behavior lives in protocol. Implementation executes protocol. They are not the same thing, and keeping them separate is the governing invariant of the runtime architecture.**

3. Runtime as a Universal Execution Substrate

The compiler paper addressed the question: *Why can governance exist before execution?*

This paper addresses the complementary question: *Why can execution exist independently of implementation?*

The answer lies in the architectural position of the runtime:



The runtime is not an application. It is a **substrate-independent execution model**: a definition of what it means to faithfully execute a Protocol Snapshot, independent of where that execution happens.

Traditional systems couple behavior to implementation. Business logic lives in code. To change the hosting environment, you port the code. To add a platform, you recompile or rewrite. Behavior is inseparable from its substrate.

PGS inverts this relationship. Behavior is declared in protocol. Protocol compiles to snapshot. Snapshot is consumed by any conforming execution substrate. The protocol travels. The substrate varies.

This has a consequence that is easy to understate: **every execution substrate that conforms to the runtime contract executes identical protocol behavior.** Not approximately identical. Not semantically equivalent. Identical. The governing topology, the admission rules, the capability declarations, the routing conditions — all of it is fixed in the snapshot before any substrate touches it.

> **The runtime is the execution contract. The substrate is the execution environment. They are orthogonal.**

4. What the Runtime Consumes

The runtime's only input from the protocol stack is the Protocol Snapshot. The snapshot is not protocol source. It is a compiled, attested, and integrity-verified artifact. The runtime never reads protocol declarations directly. It reads only from the snapshot.

The snapshot provides four logical projections to the runtime:

Tokenized Projection — the primary execution substrate. Compiled routing tables, capability handler references, input bindings, outcome conditions, and entry points — all encoded as integer addresses. No string parsing is required during execution. The execution surface is a pure integer address space. This integer-address-only representation enables both extreme execution efficiency and substrate flexibility: any conforming executor, from a cloud service to embedded firmware to a silicon core, consumes the same projection form.

Trust Projection — the attestation layer. A hash record produced by the compiler against which the runtime verifies the tokenized projection before any execution begins. If the hashes do not match, the runtime refuses to start. There is no override, no warning, no fallback. The snapshot is either fully compiler-produced and attested, or the runtime will not execute.

Vocabulary Projection — the semantic bridge. A bidirectional mapping between integer addresses and fully-qualified artifact names. Used at trace emission time to surface human-readable names for auditability. Not consulted during execution traversal — the execution surface operates entirely on integer addresses.

Visualization Projection — the compiled topology graph. Consumed by the evidence projection tool after execution to produce visual execution records. Not part of the execution path.

The tokenized projection is the operational core. Every execution decision is made by consulting it. The trust projection enforces the integrity contract before any execution begins. The vocabulary projection surfaces names in traces but plays no role in routing. The visualization projection enables post-execution evidence production.

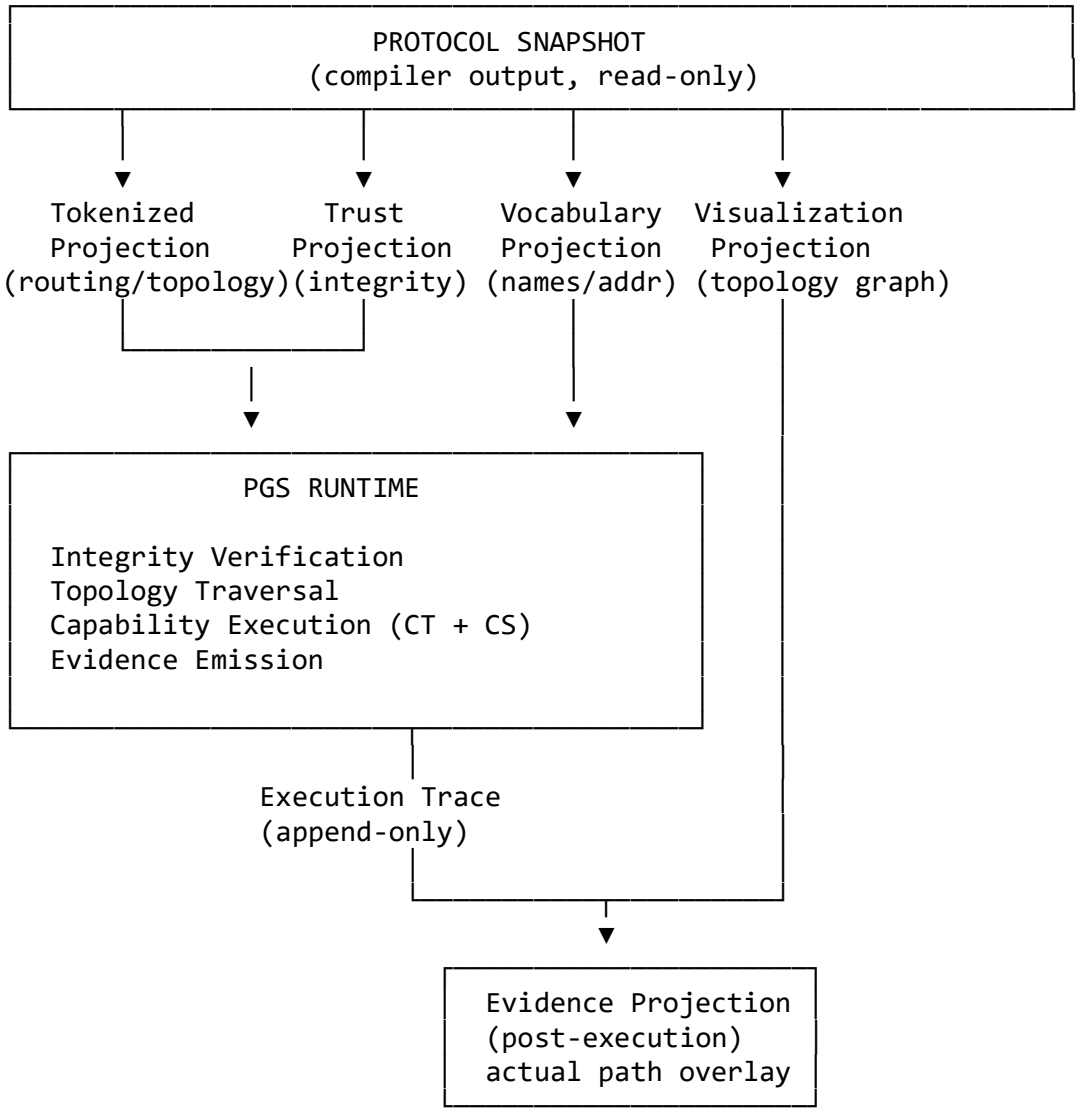


Figure 1: Runtime functional model. The runtime consumes the tokenized and trust projections to execute workflow topologies. The vocabulary projection surfaces human-readable names in execution traces. The visualization projection is consumed post-execution to overlay the actual execution path on the compiled topology. These are independent consumers of the same compiled snapshot.

Note: This figure reflects the functional architecture of the v0.4.0 PGS reference implementation. The tokenized projection shown is this implementation's primary execution substrate; other conforming runtimes may use alternative projection forms matched to their execution substrate.

5. Hosting Transparency

Traditional software couples application behavior to its hosting environment:

```
Application
    ↕ (tightly coupled)
Hosting Environment
```

To change the hosting environment — scale up, move to cloud, deploy to embedded — you must change the application. Behavior and substrate are inseparable.

PGS introduces a different relationship called **hosting transparency**: the Protocol Snapshot is completely indifferent to the substrate executing it.

```
Protocol Snapshot → Runtime A  →  Linux Server

Protocol Snapshot → Runtime B  →  Container Cluster

Protocol Snapshot → Runtime C  →  Embedded Firmware

Protocol Snapshot → Runtime D  →  FPGA Execution Engine
```

In each case, the behavior is identical. The routing topology does not change. The admission rules do not change. The capability declarations do not change. The execution evidence is structurally equivalent.

What changes across substrates:

- **Performance** — latency, throughput, resource consumption
- **Security posture** — isolation guarantees, hardware attestation
- **Cost** — compute pricing, power consumption
- **Scale** — parallelism, replication capacity

What does not change:

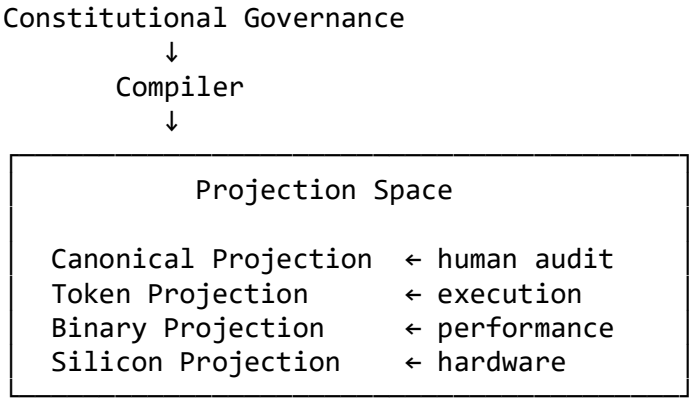
- Protocol behavior
- Execution topology
- Governance rules
- Trace semantics

This is hosting transparency. The protocol is hosted. The behavior travels. The substrate is a deployment decision, not a protocol decision.

> **A PGS deployment decision changes where execution happens. It cannot change what execution means.**

6. Projection Independence

The compiler produces multiple projections from a single governance source. Each projection is a different representation of the same compiled truth, suited to a different consumer.



All projections derive from identical governance. No projection adds behavior. No projection removes behavior. They are representation transformations of a single compiled truth.

This creates a property called **projection independence**: governance is separable from the representation used to execute it.

Traditional systems fuse governance with representation:

```
Application Code = Business Logic + Implementation + Execution Representation
```

PGS separates them:

```
Protocol  = Governance (representation-independent)
Snapshot  = Governance projected into execution representation
Runtime   = Consumes whichever projection matches the substrate
```

A token-native runtime consumes the token projection. A hardware execution unit consumes the binary projection. A silicon execution core consumes the hardware projection. Each executes identical governance — none of them are aware of the others.

This means governance can be validated once and executed everywhere. The constitutional review happens at the governance layer. The execution happens at whatever substrate is appropriate. The separation is structural, not procedural.

> **Governance does not prescribe execution representation. Execution representation does not alter governance.**

7. Security as a Projection Choice

A consequence of projection independence that deserves explicit treatment: **the security posture of an execution can be improved without changing the protocol.**

Same protocol. Different projection. Different attack surface.

```
Canonical Projection
    ↓
General-purpose runtime     ← broader execution surface
                              human-readable, interpretive

Token Projection
    ↓
Token-native runtime        ← reduced attack surface
                              no string parsing, no interpretation

Binary Projection
    ↓
Binary execution engine     ← further reduction
                              opaque to static analysis

Silicon Projection
    ↓
Hardware execution unit     ← maximum isolation
                              air-gapped from software stack
```

At each level, the attack surface decreases while protocol behavior remains invariant. There is no path to SQL injection when there is no SQL. There is no side-channel through string parsing when execution is integer-address-only. There is no software exploit surface when execution is in silicon.

This is security inversion relative to traditional systems. In traditional systems:

```
Adding security → requires changing the application
```

In PGS:

```
Improving security → requires choosing a more constrained projection
```

The application — the protocol — does not change. Only the execution substrate changes.

This has practical consequences beyond exotic hardware:

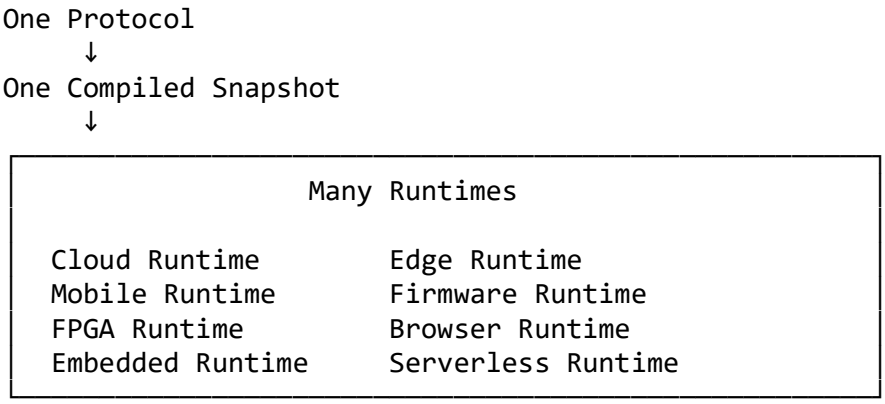
- A tokenized projection runtime cannot accidentally evaluate unescaped user input as executable code, because there is no evaluation path for free text.
- A runtime that executes sealed, compile-time-fixed capability references cannot be directed to invoke arbitrary code, because there is no discovery mechanism.

- A runtime with no routing logic of its own cannot be manipulated into alternative execution paths through injection, because routing is fixed in the compiled execution map.

In PGS, the runtime's ignorance is a security property. What the runtime cannot know, an attacker cannot exploit through it.

8. Runtime Multiplicity

A natural consequence of hosting transparency and projection independence is **runtime multiplicity**: there is no canonical single runtime.



Each runtime is a conforming implementation of the execution contract. Each executes the same protocol. None of them know about each other. All of them produce semantically equivalent execution evidence.

Traditional software portability is **implementation portability**: move the code, adapt the code, recompile the code. The goal is to make the implementation run somewhere new.

PGS portability is **protocol portability**: compile the protocol once, run it anywhere that conforms to the execution contract. No code changes. No adaptation. The snapshot is the portable artifact.

Runtime multiplicity also clarifies an architectural boundary: a runtime is not responsible for knowing how many other runtimes exist or what they are executing. It is only responsible for faithfully executing the snapshot it was given. Coordination — if any — is a governance concern declared in the protocol, not an emergent property of runtime discovery.

> **Protocol portability: compile once, execute anywhere. Not because the code is portable — because the protocol is.**

9. Why the Runtime Is Intentionally Simple

The simplicity of the runtime is not a concession. It is a structural guarantee.

By the time a workflow invocation reaches the runtime, the compiler has already resolved:

- every execution path that is constitutionally admissible
- every input binding relationship between workflow nodes
- every routing condition and continuation target
- every capability contract that a node may invoke
- every storage location and access policy a capability may use
- every permissible outcome a capability contract may declare

Nothing remains ambiguous. Nothing requires runtime interpretation. The compiler produced a flat, addressed, machine-readable execution surface. The runtime reads that surface and executes it.

This means that if behavior must change, the answer is never to change the runtime. The answer is always to change the protocol and recompile. The runtime is a constant. Protocol is the variable.

```
What changes behavior:       Protocol declarations → Compiler → Snapshot
What executes behavior:      Runtime (constant, domain-ignorant)
What records behavior:       Execution trace (append-only, deterministic)
```

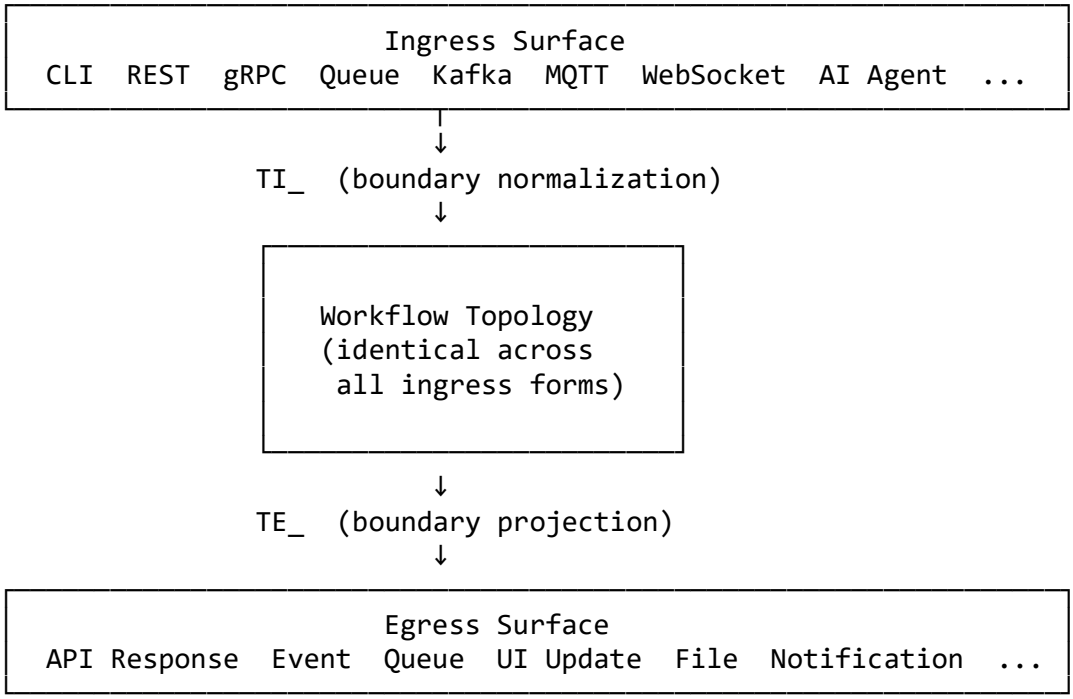
Runtime simplicity carries an important proof-theoretic consequence: **a simple runtime cannot diverge from the compiled snapshot.** If the runtime has no routing logic of its own, its routing behavior is entirely determined by what the compiler wrote. Protocol sovereignty is structural, not aspirational.

The same runtime that executes blockchain workflows executes AI governance workflows. It behaves identically in both cases because it interprets only the compiled snapshot, not domain knowledge.

> **Runtime dumbness is not a weakness. It is the proof of compiler completeness.**

10. Transport as the Universal Boundary

Every interaction with a PGS system crosses a transport boundary. Everything that enters the system enters through Transport Ingress (`TI_`). Everything that exits the system exits through Transport Egress (`TE_`). The workflow topology between these boundaries is identical regardless of how the call arrived or where the result goes.



Transport ingress normalizes the external call into the canonical internal form. A CLI invocation, a REST request, a queue message, and an AI agent callback all arrive at the workflow topology as the same normalized payload.

The topology cannot distinguish — and is constitutionally forbidden from knowing — how the call arrived. This is not merely a design convenience. It is a constitutional property. If a workflow could detect its transport origin, it could be written to behave differently depending on the caller. Transport sovereignty would be broken: governance would depend on environmental conditions rather than protocol declarations.

Transport egress projects the internal execution result to whatever external form the caller expects. The topology produces one result surface. Egress adapters project that surface into API responses, events, notifications, or queue messages.

This creates **transport orthogonality**: the transport surface is a deployment concern, not a protocol concern. Adding a new ingress adapter — say, MQTT support or an AI agent callback — requires no protocol changes, no compiler changes, no snapshot changes, and no runtime changes. It requires only a new `TI_` implementation that normalizes to the

canonical internal form. The same is true for egress: new output formats require only new `TE_` projections.

The transport boundary is also the natural home of `pgs_transport`: the package of ingress and egress adapters that connect external surfaces to the invariant workflow topology without touching protocol declarations.

> **The workflow topology is transport-neutral by construction. Transport is a boundary declaration. It is not a routing condition.**

11. Workflow Topology Traversal

A workflow execution is a directed traversal of a compiled topology graph. The runtime receives a workflow identifier and a payload. It resolves the compiled entry point and begins traversal.

Traversal is map-driven. Each step consults the compiled execution map: which capability node to execute next, given the current node and the outcome it produced. No routing logic exists in the runtime. The compiled execution map is the control flow, and the compiler wrote it.

Nodes in the topology fall into two categories:

Boundary nodes — the admission gate (`IN_`) and the terminal (`EXIT`). Boundary nodes have no capability pipeline. They represent governance checkpoints: the point at which the protocol declares that execution may proceed, or that execution has concluded.

Capability nodes — workflow capability contracts (`CC_`). Each capability node has a compiled pipeline of capability invocations. The runtime drives the pipeline, accumulates results, and returns the node's outcome to the traversal engine, which uses it to advance to the next node.

Between any two nodes, the routing condition is declared: which outcome at the current node leads to which node next. The runtime evaluates conditions by compiled map lookup. It makes no conditional decisions of its own.

Traversal terminates when the compiled execution map yields a terminal node. At that point, the workflow result is the accumulated surface from the final capability node. The inter-CC data flow is the only form of cross-node communication in PGS: explicit, compiler-declared, path-addressed binding. No ambient state passes between nodes.

Workflow executions are isolated from one another by construction. One execution's results cannot influence another's topology. This is not a concurrency policy imposed at runtime. It is a structural property: the execution context is local to each invocation.

12. The Two Capability Kinds

The nine PGS execution concerns separate behavior into two capability categories that the runtime treats very differently.

CT — Capability Transform (pure computation)

A Capability Transform is a pure function. Given the same inputs, it always produces the same outputs. It has no side effects. It does not interact with external state. It does not make network calls. It does not access the filesystem. It does not read from or write to the world outside the execution context.

This purity is not a convention. It is a constitutional invariant. A CT may never invoke a CS. The compiler enforces this at construction time. The runtime enforces it structurally: the CT execution path has no route to the CS executor.

CT correctness is the compiler's responsibility. The compiler produces and verifies the CT specification before attesting the snapshot. The runtime executes what the compiler produced. CT failure is a protocol violation — a governance signal — not a runtime error to be handled.

CS — Capability Side Effect (controlled external interaction)

A Capability Side Effect is a controlled interaction with external state. File writes, registry mutations, event appends, external API calls — any interaction with the world outside the execution context is a CS.

CS behavior is bounded and enumerated. Each CS declares the operations it supports. The runtime invokes only declared operations. An undeclared operation cannot be invoked — it is not present in the compiled handler specification.

CS instantiation is sealed at compile time. The compiler emits a handler specification containing the implementation reference. The runtime instantiates the CS from this sealed specification. This is the single point of dynamic dispatch in the runtime, and it is sealed by the compiler, not discovered at execution time.

The capability configuration — storage locations, connection parameters, access policies — comes from the Runtime Binding (`RB_`) artifact compiled into the snapshot. This keeps deployment-specific paths out of the protocol while keeping them declared and auditable.

The distinction between CT and CS is load-bearing. It declares, with governance authority, which executions are pure and which produce effects. The runtime inherits this distinction from the compiled snapshot.

13. Parallelism as a Structural Property

Traditional software systems derive concurrency from implementation choices: thread pools, async frameworks, lock disciplines, queue depths. Concurrency is an engineering concern imposed on top of the business logic.

PGS inverts this relationship. Concurrency is a property of the protocol topology, not of the runtime implementation.

The structural reasons are:

Workflow independence — workflows share no mutable state. Two workflow executions from the same snapshot cannot interfere with each other through the execution topology. Isolation requirements, if any, are declared in capability side effects.

Capability node isolation — capability contract results are accumulated locally within each execution. A node's output is visible only to the workflow that invoked it, via explicitly declared bindings.

CT purity — pure capability transforms are unconditionally parallelizable. There are no shared resources to coordinate. Same inputs always produce the same outputs regardless of concurrency.

Outcome-driven routing — traversal advances when an outcome is available. The runtime does not block waiting for external state changes. Control flow is event-driven.

Non-blocking boundaries — transport ingress and egress are boundary projections, not synchronization points within the workflow.

The runtime scheduler does not create concurrency. It exploits concurrency already declared by the topology. This distinction matters: parallelism in PGS is not a runtime optimization applied to sequential logic. It is a structural property of the protocol that the runtime inherits. A runtime that implements concurrent node execution, distributed workflow dispatch, or actor-model execution does not need to discover which workflows can run in parallel. The compiled execution map declares what is structurally independent. The runtime exploits it.

```
Traditional:
  Implementation decides what can run concurrently.

PGS:
  Topology declares what is structurally independent.
  Runtime exploits that independence to whatever degree the substrate permit
s.
```

A cloud runtime might execute thousands of independent workflow invocations concurrently. A firmware runtime might execute one at a time. Both are correct. The protocol declares independence. The substrate determines exploitation.

Concurrency is not engineered into PGS. It is a structural consequence of protocol topology.

14. Runtime as a Deterministic Appliance

A useful frame for understanding the PGS runtime is to compare it to well-understood execution appliances:

- A **database engine** accepts queries. It does not know the business purpose of the query. It executes the declared operation faithfully and produces a result.
- A **JVM** accepts bytecode. It does not know what the bytecode computes. It executes the declared instructions faithfully and produces a result.
- A **web browser** accepts markup and script. It does not know the purpose of the page. It renders and executes the declared resources faithfully.

The PGS runtime occupies the same architectural position:

- It accepts a Protocol Snapshot. It does not know the business domain. It executes the declared topology faithfully and produces execution evidence.

This framing clarifies what users of PGS build. They build **protocols**, not runtimes. The runtime is infrastructure — a fixed, governed, deterministic appliance. Improving the protocol does not require touching the runtime. Adding new domains does not require changing the runtime. Deploying to a new hosting environment does not require rewriting the runtime — it requires conforming the new substrate to the execution contract.

```
Users build:          Protocol
Compiler builds:      Snapshot
Runtime does:         Execute the snapshot faithfully
Runtime does not do:  Domain logic
```

This is a fundamental shift from traditional software development, where business logic and execution engine are developed in tandem. In PGS, they are entirely separate concerns. The execution appliance is a stable, versioned infrastructure component. The protocol is the artifact that evolves with the domain.

15. Runtime Stability

Implementation independence produces a property with direct engineering consequence: **the runtime stabilizes before the protocol stabilizes.**

In traditional systems, the application grows with the domain. Every new business requirement touches the codebase. The execution engine, the business logic, and the domain model evolve together. Stability is deferred until the domain is fully understood.

In PGS, the relationship is inverted:

```
New domains                → protocol work (compiler + snapshot)
New workflows              → protocol work (compiler + snapshot)
New capability contracts   → protocol work (compiler + snapshot)
New governance rules       → protocol work (compiler + snapshot)

None of the above          → runtime changes
```

The runtime reaches a stable state when its execution contract is correct. After that point, it is infrastructure: version-locked, testable in isolation, deployable without domain knowledge. New protocols compile to new snapshots. The runtime executes them without modification.

This is a major consequence for system evolution. Traditional systems require coordinated changes across business logic and execution engine. PGS requires only protocol changes, which the compiler validates independently of the runtime. The runtime is the stable substrate against which protocol changes are tested.

It is also a consequence for security review. A stable, audited runtime is a contained scope. The attack surface of the runtime does not grow as the protocol grows. New domains do not introduce new runtime code paths.

> **In PGS, the runtime is a stable foundation. Protocol evolution does not destabilize the executor.**

16. Determinism

The runtime makes a strong guarantee: identical inputs produce identical execution paths and identical traces.

This guarantee holds structurally:

- The snapshot is immutable and integrity-verified before execution begins
- The compiled execution map is a pure function from (current node, outcome) to next node — there are no runtime choices
- Capability Transforms are pure functions — same inputs always produce same outputs
- Capability Side Effects declare their operations — non-determinism, if any, is bounded inside the side effect implementation, not inside the routing logic
- Execution evidence is derived entirely from execution events — the same execution produces the same trace content

The consequence is observable: given the same snapshot and the same payload, the execution path and the execution trace will be identical across every conforming runtime on every conforming substrate.

This observable determinism is the basis of compliance verification, regression detection, and protocol auditing. It is not a property that needs to be engineered into each deployment. It is a property of the architecture.

17. Trace Portability

If two runtimes execute the same Protocol Snapshot with the same payload — a Linux server runtime and an embedded firmware runtime, for example — the execution traces they produce are **semantically equivalent**.

The artifact identities are identical. The routing paths are identical. The outcome sequences are identical. The capability invocations are identical in name and declaration. What differs is substrate-local: wall-clock timestamps, physical storage locations, performance measurements.

This is **trace portability**: execution evidence is substrate-neutral.

Trace portability has concrete operational consequences:

Cross-substrate verification — you can compare the execution behavior of a cloud runtime against an embedded runtime executing the same protocol. If the traces diverge in semantics, the divergence is a governance violation, not a configuration difference.

Protocol regression testing — traces from a previous snapshot version can be compared against traces from a new snapshot version. Behavioral changes are detectable in the evidence record without access to the substrate.

Compliance auditing — a regulatory audit of protocol execution does not require access to the specific substrate that produced the trace. The trace is the evidence. Its provenance is the snapshot attestation, not the runtime identity.

Multi-runtime certification — a new runtime implementation can be certified against a reference runtime by executing the same snapshot and comparing trace semantics. The snapshot is the test suite. The traces are the test results.

> **The trace is not an artifact of a particular runtime. It is an artifact of a particular protocol execution.**

18. Execution Evidence

Every workflow execution produces a structured, append-only execution trace. The trace records every governance event: workflow start, capability node entry, individual capability step completion, node completion, and workflow termination. Each event carries the execution identity, the domain, and the artifact identifiers involved. Artifact names appear in human-readable form alongside their integer addresses for auditability.

The trace identifier is derived deterministically from the workflow identity and payload. The same inputs produce the same identifier suffix, enabling duplicate detection across executions without enforcing strict uniqueness.

The trace is the authoritative execution record. It is immutable once written. The runtime never reads its own traces. Traces are consumed by external tools: diagnostic analysis and evidence projection.

Evidence projection is a post-execution operation that combines the execution trace with the compiled topology graph to produce a visual execution record. It overlays the actual execution path on the full compiled topology, distinguishing taken paths from untaken alternatives. This is not part of the execution path. It is a post-hoc visualization of materialized evidence — the two faces of the same execution truth, one structured for machine processing and one rendered for human review.

Both derive from the same authoritative source: the append-only execution log written by the runtime during traversal, and the compiled graph written by the compiler before any execution began.

19. Runtime Boundary Contract

What the runtime knows

- The compiled execution map: topology, routing conditions, input bindings, entry points
- The compiled capability handler specifications and their configurations
- The vocabulary: bidirectional mapping between integer addresses and artifact names
- The invocation payload
- The data root for capability configuration expansion

What the runtime does not know

- Protocol declarations (source governance documents)
- Constitutional rules, federation boundaries, or invariants
- Domain semantics — what an actor, wallet, license, or agent action means
- Compiler internals or construction logic
- The execution state of any other workflow invocation
- How many other runtimes exist or are executing simultaneously

What the runtime cannot do

- Alter the execution topology at runtime — routing is fixed in the compiled execution map
- Infer missing configuration — missing artifact means hard failure, no default
- Invoke a capability not declared in the snapshot — the execution surface is sealed at compile time
- Suppress execution evidence — traces are unconditional
- Execute an unattested snapshot — integrity mismatch means refusal before any execution begins

These prohibitions are structural, not conventional. The runtime's execution path has no code that could perform them.

20. The Execution Concerns in Runtime

The nine PGS execution concerns map to runtime behavior as follows:

Concern	Prefix	Runtime Role
Transport Ingress	`TI_`	Boundary normalization — normalizes external calls to canonical internal form; handled by transport layer, not core runtime
Actor Context	`AC_`	Authority binding — compiled into capability configurations and binding policies
Intent	`IN_`	Admission gate — boundary node in topology; declares the governance checkpoint for execution to proceed
Workflow	`WF_`	Topology — the traversal engine follows the compiled execution map for this workflow
Capability Contract	`CC_`	Named node — the execution engine drives the compiled capability pipeline for this node
Capability Transform	`CT_`	Pure computation — evaluated with zero side effects; constitutional purity invariant
Capability Side Effect	`CS_`	Controlled interaction — instantiated from sealed handler reference and invoked with declared operation
Event	`EV_`	Governance signaling — emitted as declared values in capability inputs; not interpreted by runtime
Transport Egress	`TE_`	Boundary projection — the result surface returned by the runtime to the transport layer

Orthogonal resolution:

Concern	Prefix	Runtime Role
Runtime Binding	`RB_`	Configuration — provides the capability configuration the runtime expands with the data root at execution time

The runtime consumes all nine concerns as compiled data. It interprets none of them as semantic objects. Event values, for example, appear as declared strings in capability input bindings. The runtime passes them as payload fields. Their governance significance was resolved by the compiler. The runtime simply carries the value forward.

21. Conclusion

The PGS runtime is the smallest possible executor consistent with the governed execution model. It reads a compiled, attested, integrity-verified execution map and executes it faithfully. It contains no domain logic, no inference, no fallback, and no interpretation.

But understanding only that the runtime is simple misses the larger consequence.

Traditional systems make behavior portable by moving implementations. Code is ported, adapted, recompiled. The implementation is the unit of portability, and it must be carried everywhere behavior is needed.

PGS makes behavior portable by moving protocol. The protocol is compiled to a snapshot. The snapshot executes on any conforming substrate. The implementation is not the unit of portability — the protocol is. The runtime is merely the executor of a portable governance artifact.

This produces the properties that define PGS execution:

- **Implementation independence** — behavior lives in protocol; implementation can be replaced without changing governance
- **Substrate independence** — the same snapshot executes identically on Linux, containers, embedded firmware, FPGAs, and browsers
- **Hosting transparency** — the substrate is a deployment decision; it cannot alter protocol behavior
- **Projection independence** — governance is separable from execution representation; security posture improves by choosing a more constrained projection
- **Runtime multiplicity** — one protocol, many conforming runtimes, all executing identical behavior
- **Transport orthogonality** — the workflow topology is constitutionally ignorant of its invocation surface
- **Structural parallelism** — concurrency is declared by topology, not engineered into implementation
- **Runtime stability** — the runtime reaches a stable state; protocol evolution does not touch the executor
- **Trace portability** — execution evidence is substrate-neutral and comparable across radically different hosting environments

The runtime is the execution contract. The Protocol Snapshot is the governed artifact it executes. Everything that needs to be true about behavior was resolved by the compiler before the runtime was ever invoked.

The compiler governs possibility. The runtime governs realization. The separation between them is where Protocol-Governed Systems become governable.

Appendix A: Key Terms

Protocol Snapshot: The compiler's primary output. A complete, verified description of the admissible execution surface of the system. Read-only input to the runtime. Never modified by hand.

Tokenized Projection: A machine-optimized projection of the protocol topology in which all artifact identifiers are replaced by compact numeric tokens derived from their fully qualified names at compile time. Enables execution on constrained, embedded, and silicon substrates.

Trust Projection: A compiler-produced attestation record containing the integrity hash of the tokenized projection. The runtime verifies this hash before any execution begins. Mismatch causes unconditional refusal.

Vocabulary Projection: A bidirectional mapping between numeric addresses and fully-qualified artifact names. Used at trace emission time to surface human-readable names. Not consulted during execution traversal.

Visualization Projection: A compiler-produced graph of the workflow topology. Consumed post-execution to overlay the actual execution path on the compiled topology. Not part of the execution path.

Implementation Independence: The property that protocol behavior can be executed without coupling to a specific implementation. The implementation can be replaced without changing governance; the protocol can evolve without changing the runtime.

Hosting Transparency: The property that the Protocol Snapshot is indifferent to the substrate executing it. Behavior is identical across substrates; only performance, security posture, and cost characteristics vary.

Projection Independence: The property that governance is separable from the execution representation. Multiple projections can be produced from the same compiled truth; selecting a projection does not alter the governed behavior.

Security as a Projection Choice: The architectural property that security posture improves by selecting a more constrained execution projection — without changing the protocol. A token-native runtime has a smaller attack surface than an interpretive runtime executing the same governance.

Runtime Multiplicity: The architectural property that one protocol can be executed by many conforming runtimes simultaneously — cloud, edge, embedded, hardware — all producing semantically equivalent execution evidence.

Transport Orthogonality: The constitutional property that the workflow topology cannot distinguish between ingress forms. A workflow executed via CLI and the same workflow executed via REST API are indistinguishable at the topology level.

Runtime Stability: The property that the runtime reaches a stable execution contract before the protocol stabilizes. Protocol evolution produces new snapshots; the runtime executes them without modification.

Trace Portability: The property that execution evidence is substrate-neutral. Traces produced by a cloud runtime and an embedded firmware runtime executing the same snapshot are semantically equivalent and comparable.

Execution Concern: One of the nine named categories into which all protocol artifacts are classified (TI, AC, IN, WF, CC, CT, CS, EV, TE), plus the orthogonal Runtime Binding (RB). The runtime treats each concern category differently but has no semantic knowledge of any of them.

CT / CS Distinction: The constitutional separation between Capability Transforms (pure computation, zero side effects) and Capability Side Effects (controlled external interactions). This separation is a compile-time invariant enforced structurally in the runtime execution path.

Fully Qualified Domain Name (FQDN): The canonical identity of a protocol artifact. Format: `domain::ARTIFACT_CODE_Vn`. Surfaces in execution traces for human readability; not used during execution traversal.

Governance Dividend: The observable property that system evolution becomes more localized as governance matures. The runtime remains unchanged as the protocol evolves; new domains do not introduce new runtime code paths.

Appendix B: Reference Implementation Notes

The conceptual model presented in this paper has been realized in the open-source Protocol-Governed Systems (PGS) reference implementation available on GitHub:

PGS Workspace Repository

The implementation serves as a practical realization of the concepts discussed throughout this paper, demonstrating how a domain-ignorant runtime can execute governed protocol behavior across multiple domains — blockchain identity, wallet management, transaction submission, AI agent governance, and AI licensing — from a single compiled snapshot.

The examples, terminology, and architectural discussions in this paper are based on **PGS v0.4.0**, which represents the state of the project at the time of publication. The v0.4.0 runtime implements the tokenized projection as its primary execution substrate, produces append-only JSONL execution traces, and generates visual execution path overlays via post-execution evidence projection.

Since PGS is under active development, subsequent releases may introduce additional projection forms, extended transport adapters, enhanced admission gating, alternative execution substrates, and further runtime capabilities beyond those described here. The conceptual properties documented in this paper — implementation independence, hosting transparency, projection independence, runtime multiplicity, transport orthogonality, structural parallelism, runtime stability, and trace portability — are architectural properties of the PGS model, not features specific to any release.

For the latest documentation, releases, and implementation details, consult the project repository.

Appendix C: References

Ganti, B. (2026). *Protocol-Governed Systems: Conceptual Model*. DOI: https://doi.org/10.5281/zenodo.20300611

Ganti, B. (2026). *Protocol-Governed Systems: Compiler Conceptual Model*. DOI: https://doi.org/10.5281/zenodo.20471804

Lamport, L. (1994). The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems*, 16(3), 872–923.

Lee, E. A. (2006). The Problem with Threads. *IEEE Computer*, 39(5), 33–42.

Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308.